

STM 32- 3. HAFTA IŞIL VE NISA SUNAR

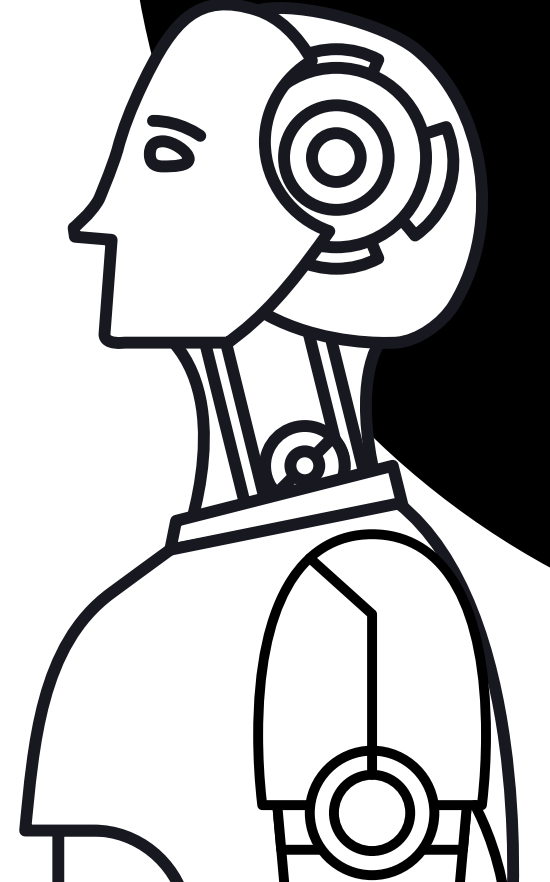
I2C (INTER-INTEGRATED CIRCUIT) NEDİR?

Philips (şimdi NXP) tarafından geliştirilmiş bir seri haberleşme protokolü.

Amaç: Mikrodenetleyici (master) ile sensör, EEPROM, ADC gibi çevre birimleri (slave) arasında veri iletimi

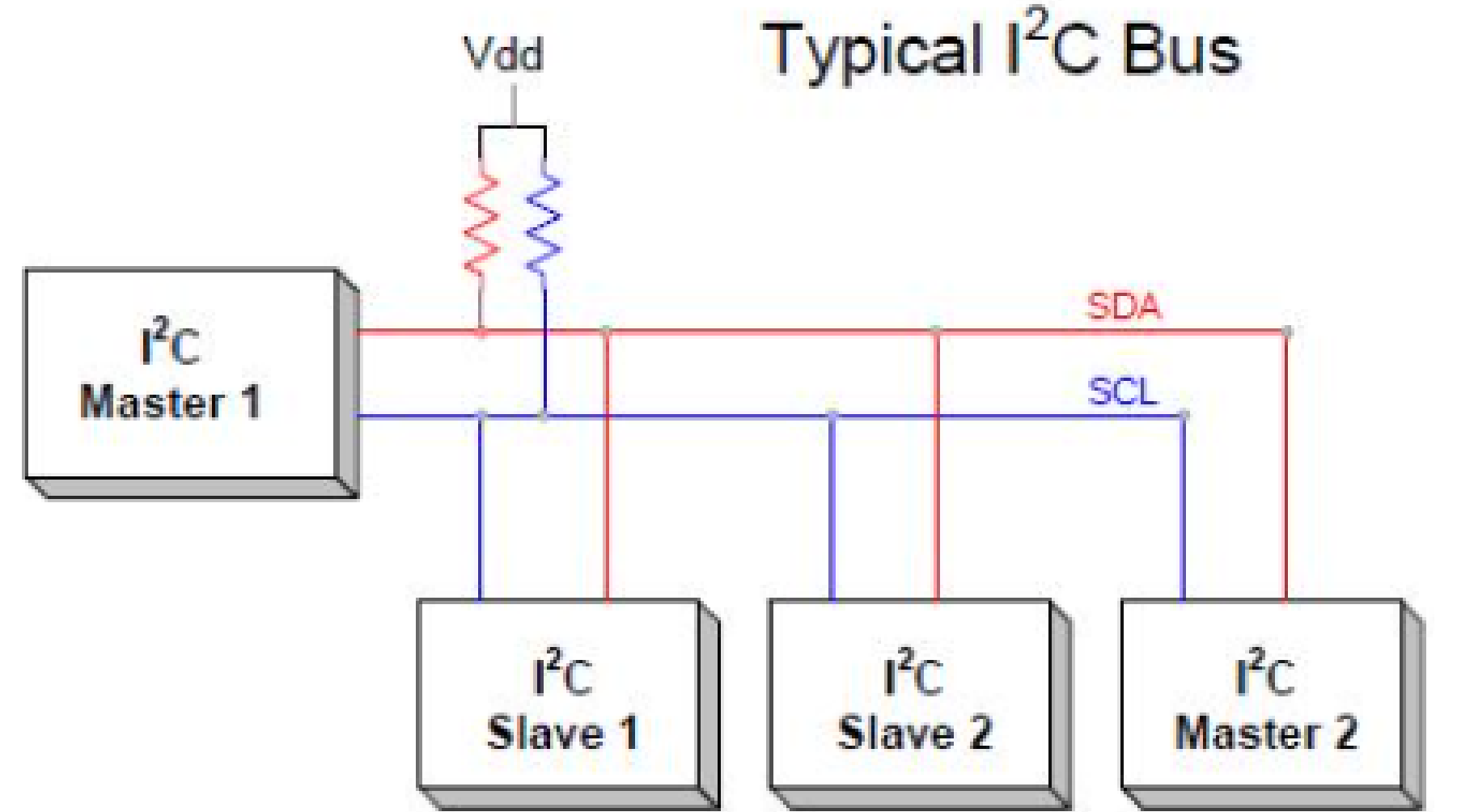
Özellikleri:

- **İki hat kullanır:**
 - SDA (Data Line) → veri
 - SCL (Clock Line) → saat
- **Multi-master ve multi-slave destekler.**
- **Adresleme yapılır: 7-bit veya 10-bit adres.**
- **Senkron çalışır (veri, saat sinyaliyle hizalanır).**



I²C HATTI FİZİKSEL YAPISI

- Tüm cihazlar aynı SDA ve SCL hattına bağlıdır.
- Pull-up dirençleri gerekir (genellikle 4.7k Ω veya 10k Ω).
- Hat yüksek (1) olduğunda veri aktarımı yok, düşük (0) olduğunda veri aktarılır.



I²C TEKNİK ÇALIŞMA MANTIĞI

PAGE 03

01

BAŞLATMA VE DURDURMA

- Start condition (S):
- SDA, SCL yüksekken → SDA düşer → veri başlıyor.
- Stop condition (P):
- SCL yüksekken → SDA yükselir → veri bitiyor.

SDA: ———___/—————

SCL: ———/___\—————

02

VERİ AKTARIMI

- Her byte (8 bit) saat darbeleriyle gönderilir.
- Gönderen: master veya slave
- Alıcı: master veya slave
- Her byte'tan sonra ACK (acknowledge) bit gönderilir:
 - ACK = alındı
 - NACK = alınmadı

03

ADRESLEME

- Her slave'in benzersiz adresi vardır (7 veya 10 bit).
- Master önce slave adresini + R/W biti yollar.

Slave adresi uyuşuyorsa ACK verir ve veri aktarımı başlar.

I²C MODLARI

- **Standard mode:** 100 kHz
- **Fast mode:** 400 kHz
- **Fast Plus:** 1 MHz
- **High-speed mode:** 3.4 MHz

ÖRNEK TEKNİK AKIŞ

Master, sensörden veri okumak istiyor:

1. **Master:** Start condition
2. **Master:** Slave adresi + R bit (Read)
3. **Slave:** ACK
4. **Slave:** Veri byte'ı gönderir
5. **Master:** ACK (daha veri varsa) veya NACK (son byte)
6. **Master:** Stop condition

PASS MODE VE MULTIPLEXER

I²C hattı üzerinde çok sayıda cihaz varsa, bazen adres çakışması olur.

- Multiplexer (ör: TCA9548A) kullanılır.
- Pass mode = veriyi direkt geçiren mod (master direkt slave'e ulaşır).
- Teknik olarak: "I²C sinyalleri switch üzerinden kesilmeden aktarılır."

NELER TEKRAR EDİLMELİ / UYGULAMA

- Start / Stop condition ve veri akışı şemaları çizilebilir
- Adresleme ve ACK/NACK mantığı anlaşılmalı
- Multiplexer ve pass mode datasheet örnekleri incelenmeli

STM32 gibi bir MCU'da I²C kodu yazıp veri okuma/yazma yapılmalı

I2C'DE PASS MODE NEDİR?

Bu terim genellikle I2C çoklayıcı (multiplexer) veya switch entegrelerinde geçer.

Örneğin TCA9548A, PCA9546A gibi entegreler, bir tane I2C hattını birden fazla alt cihaza yönlendirmeye yarar.

- “pass mode”
- “bypass mode”
- “pass-through”
- “transparent mode”

Bu kısım genellikle “Operating Modes” veya “Functional Description” başlıkları altında anlatılır.

Orada şu tarz bir ifade görürsün:

“In pass-through mode, the I²C bus lines are connected directly from input to output.”

Bu da “giriş ve çıkış doğrudan bağlanır, veri geçişine izin verilir” anlamına gelir.

Normalde bu entegreler şöyle çalışır:

- Ana hattın (master cihazın) sinyali sadece seçilen kanaldan geçer.
- Diğer kanallar kapalıdır.

Ama **pass mode (veya bypass mode) açıldığında:**

- Çip, sinyali hiç engellemeden doğrudan geçirir.
- Yani **çip “arada yokmuş” gibi davranır.**
- Bu sayede master cihaz, sanki o entegre hiç yokmuş gibi diğer cihazlarla doğrudan iletişim kurar.



IN-18 DIFFERENTIAL VE SINGLE ENDED NEDİR?

PAGE 06

Bu ifade genellikle ADC giriş türünü anlatır.

ADC girişleri single-ended veya differential olabilir:

- **Single-ended:** Tek giriş toprağa göre ölçülür (örneğin A0 - GND).
- **Differential:** Ölçüm iki pin arasındaki gerilim farkına göre yapılır (örneğin IN+ ve IN-).

Yani “IN-18 Differential” diyorsa:

- ADC'nin 18 numaralı kanalının diferansiyel giriş olarak çalıştığı anlamına gelir.
- ADC, o kanalda iki pini kıyaslayarak farkı ölçer.
- **Bu yöntem, gürültü azaltmak ve hassas ölçüm yapmak için tercih edilir.**

–Biz ADC ile internal sensörde single ended yaptık çünkü tek pinimiz vardı.

ADC'DE NEDEN RIGHT ALIGNMENT YAPTIK?

PAGE 07

ADC dönüşüm sonucu örneğin 12 bit olabilir ama veri kaydı 16 bitlik bir register'a yazılır.

Bu durumda:

- **Right alignment (sağa yaslama):** En düşük anlamlı bitler (LSB) sağda olur.
 - Kolayca uint16_t türünde okunabilir.
 - Matematiksel işlem yapmak (örneğin ortalama almak) kolaylaşır.
- **Left alignment (sola yaslama):** LSB kısmı boş kalır, MSB'ye doğru kayar.
 - Genellikle 8-bit veri kullanımı gerekiyorsa tercih edilir.

Biz right alignment seçtik çünkü genellikle:

- Tam 12 bitlik veriyi 16 bit içinde işliyoruz 4 bit kaydırma HAL ve II kütüphaneleri için uygun olanı bu. Maskleme ve shifting yapmaya gerek kalmaz.
- Yazılımda kaydırma yapmaya gerek kalmadan doğru değeri elde ediyoruz.

SAMPLING CYCLE NEDEN 640.5?

Bu, **ADC'nin giriş sinyalini örnekleme süresidir** (sampling time).

Yani ADC, giriş gerilimini ölçmeden önce o değeri kondansatöre ne kadar süreyle tutar.

total adc conversation time(sonuç)=conversation time(quantization)+sampling time(kondansatöre dolma süresi)

- Eğer sensörün çıkış empedansı yüksekse (örneğin yavaş tepki veren bir sensör), **örnekleme süresi uzun olmalıdır** — böylece ADC girişi kararlı bir voltaj görebilir.
- STM32 gibi mikrodenetleyicilerde, ADC sampling time tabloda “ADC_SAMPLETIME_640CYCLES_5” olarak geçer.

Örneğin ADC saatin 80 MHz olduğunu düşünelim:

$$\text{ADC sample time} = (640.5 + 12.5) / 80 \text{ MHz} \approx 8.2 \mu\text{s}$$

Bu, yüksek empedanslı sensörler için güvenli bir ölçüm süresidir.

Kısacası: “640.5” seçilmesinin nedeni, ölçümün kararlı olması ve gürültüyü azaltmaktır.



ELEKTRONIKTE OFFSET NEDİR?

Offset, bir ölçümün veya sinyalin sıfır noktasından kayma miktarıdır.

- ADC'de: Gerçek 0 V ölçmek isterken örneğin 0.02 V okunuyorsa → offset error vardır.
- Operasyonel amplifikatörlerde (op-amp): Giriş farkı 0 V iken bile çıkışta bir voltaj varsa buna input offset voltage denir.
- Data sheette 130 ve 30 derece üzerinden hesaplama yapıldığı için offset kullanarak orijinal değere ulaşıyoruz.

Uygulamada offset:

- Kalibrasyonla düzeltilir,
- Ya da yazılımsal olarak çıkarılır (örneğin sensör_ölçüm - offset).

